

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
df=sns.load_dataset("titanic")
```

```
print(df)
```

```

   survived  pclass    sex  age  sibsp  parch   fare embarked  class \
0         0      3  male  22.0    1     0   7.2500         S   Third
1         1      1 female  38.0    1     0  71.2833         C   First
2         1      3 female  26.0    0     0   7.9250         S   Third
3         1      1 female  35.0    1     0  53.1000         S   First
4         0      3  male  35.0    0     0   8.0500         S   Third
..      ...    ...    ...    ...    ...    ...    ...    ...    ...
886        0      2  male  27.0    0     0  13.0000         S  Second
887        1      1 female  19.0    0     0  30.0000         S   First
888        0      3 female   NaN    1     2  23.4500         S   Third
889        1      1  male  26.0    0     0  30.0000         C   First
890        0      3  male  32.0    0     0   7.7500         Q   Third

```

```

   who  adult_male  deck  embark_town  alive  alone
0   man         True   NaN  Southampton    no   False
1  woman        False    C   Cherbourg   yes   False
2  woman        False   NaN  Southampton   yes    True
3  woman        False    C   Southampton   yes   False
4   man         True   NaN  Southampton    no    True
..      ...    ...    ...    ...    ...    ...
886  man         True   NaN  Southampton    no    True
887 woman        False    B   Southampton   yes    True
888 woman        False   NaN  Southampton    no   False
889  man         True    C   Cherbourg   yes    True
890  man         True   NaN  Queenstown    no    True

```

```
[891 rows x 15 columns]
```

```
#basic operations
print("Top data in the titanic data\n")
print(df.head())
print("Least of the data in the titanic data sets")
print(df.tail())
print("\n\nshape of the titanic data sets")
print(df.shape)
```

```
Top data in the titanic data
```

```

   survived  pclass    sex  age  sibsp  parch   fare embarked  class \
0         0      3  male  22.0    1     0   7.2500         S   Third
1         1      1 female  38.0    1     0  71.2833         C   First
2         1      3 female  26.0    0     0   7.9250         S   Third
3         1      1 female  35.0    1     0  53.1000         S   First
4         0      3  male  35.0    0     0   8.0500         S   Third

```

```

   who  adult_male  deck  embark_town  alive  alone
0   man         True   NaN  Southampton    no   False
1  woman        False    C   Cherbourg   yes   False
2  woman        False   NaN  Southampton   yes    True
3  woman        False    C   Southampton   yes   False
4   man         True   NaN  Southampton    no    True

```

```
Least of the data in the titanic data sets
```

```

   survived  pclass    sex  age  sibsp  parch   fare embarked  class \
886        0      2  male  27.0    0     0   13.00         S  Second
887        1      1 female  19.0    0     0   30.00         S   First
888        0      3 female   NaN    1     2   23.45         S   Third
889        1      1  male  26.0    0     0   30.00         C   First
890        0      3  male  32.0    0     0    7.75         Q   Third

```

```

   who  adult_male  deck  embark_town  alive  alone
886  man         True   NaN  Southampton    no    True
887 woman        False    B   Southampton   yes    True
888 woman        False   NaN  Southampton    no   False
889  man         True    C   Cherbourg   yes    True
890  man         True   NaN  Queenstown    no    True

```

```
shape of the titanic data sets
(891, 15)
```

```
# statistics using age
print(df.describe())
print("\n\n max age \n\n",df['age'].max())
print("-----\n\n")
print("min age \n\n",df['age'].min())
```

```

print("-----\n\n")
print("mean age \n\n",df['age'].mean())
print("-----\n\n")
print("median age \n\n",df['age'].median())
print("-----\n\n")
print("standard deviation age \n\n",df['age'].std())
print("-----\n\n")

```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

max age

80.0

min age

0.42

mean age

29.69911764705882

median age

28.0

standard deviation age

14.526497332334044

#finding missing values

df['age'].mean()

np.float64(29.69911764705882)

df['age'].fillna(df['age'].mean(),inplace=True)

/tmp/ipykernel_401/1492264711.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

df['age'].fillna(df['age'].mean(),inplace=True)

df['age']

```
      age
0  22.000000
1  38.000000
2  26.000000
3  35.000000
4  35.000000
...      ...
886 27.000000
887 19.000000
888 29.699118
889 26.000000
890 32.000000
891 rows × 1 columns
```

dtype: float64

```
df['alive']
```

```
      alive
0        no
1        yes
2        yes
3        yes
4        no
...      ...
886       no
887       yes
888       no
889       yes
890       no
891 rows × 1 columns
```

dtype: object

```
df['sex']
```

```
      sex
0    male
1  female
2  female
3  female
4    male
...     ...
886  male
887  female
888  female
889  male
890  male
891 rows × 1 columns
```

dtype: object

```
df['class']
```

```

      class
0      Third
1      First
2      Third
3      First
4      Third
...      ...
886     Second
887      First
888      Third
889      First
890      Third
891 rows × 1 columns

```

dtype: category

```
df[['age', 'sex', 'alive', 'class', 'fare']]
```

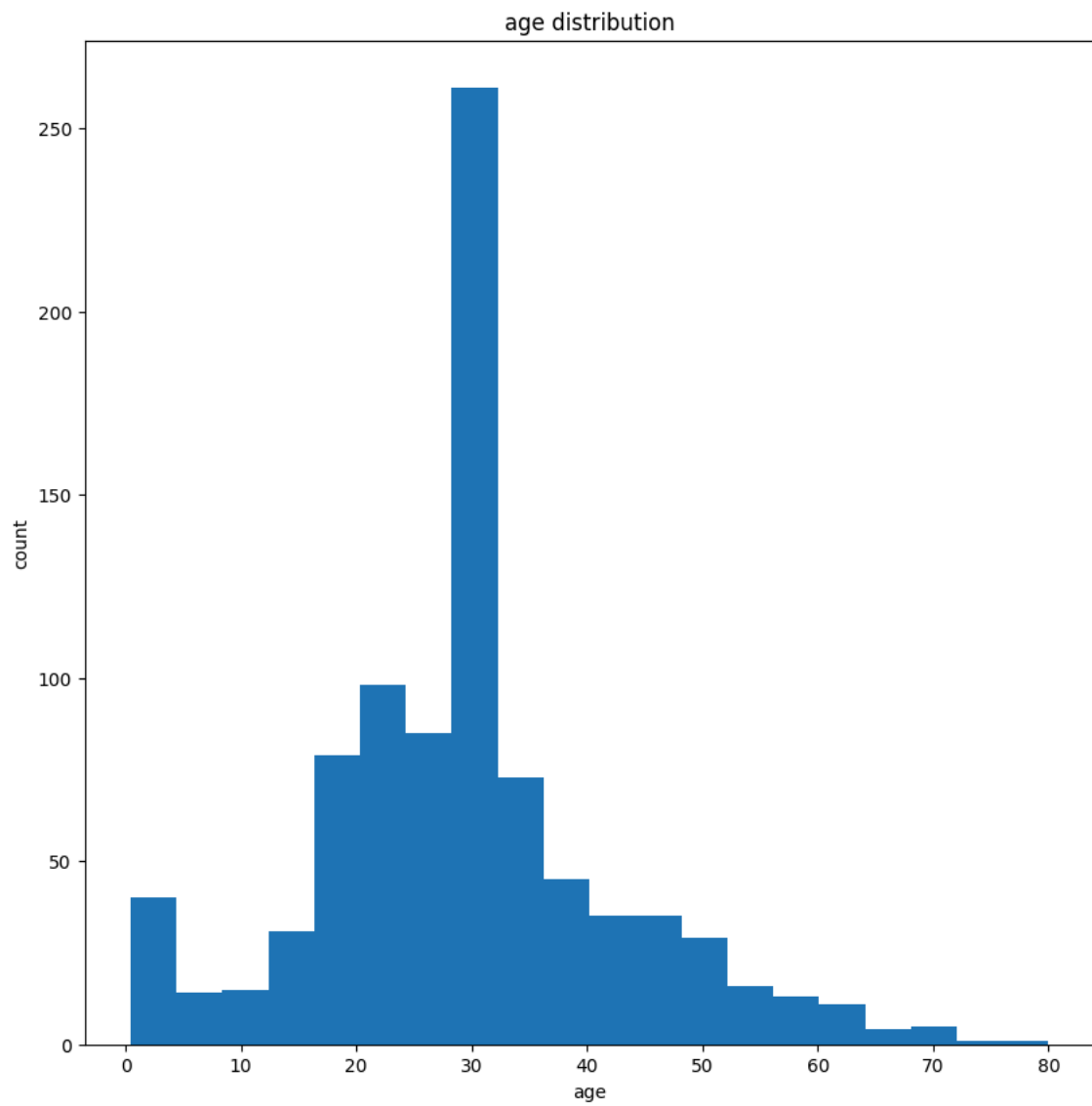
	age	sex	alive	class	fare
0	22.000000	male	no	Third	7.2500
1	38.000000	female	yes	First	71.2833
2	26.000000	female	yes	Third	7.9250
3	35.000000	female	yes	First	53.1000
4	35.000000	male	no	Third	8.0500
...	...	...	...	...	...
886	27.000000	male	no	Second	13.0000
887	19.000000	female	yes	First	30.0000
888	29.699118	female	no	Third	23.4500
889	26.000000	male	yes	First	30.0000
890	32.000000	male	no	Third	7.7500

891 rows × 5 columns

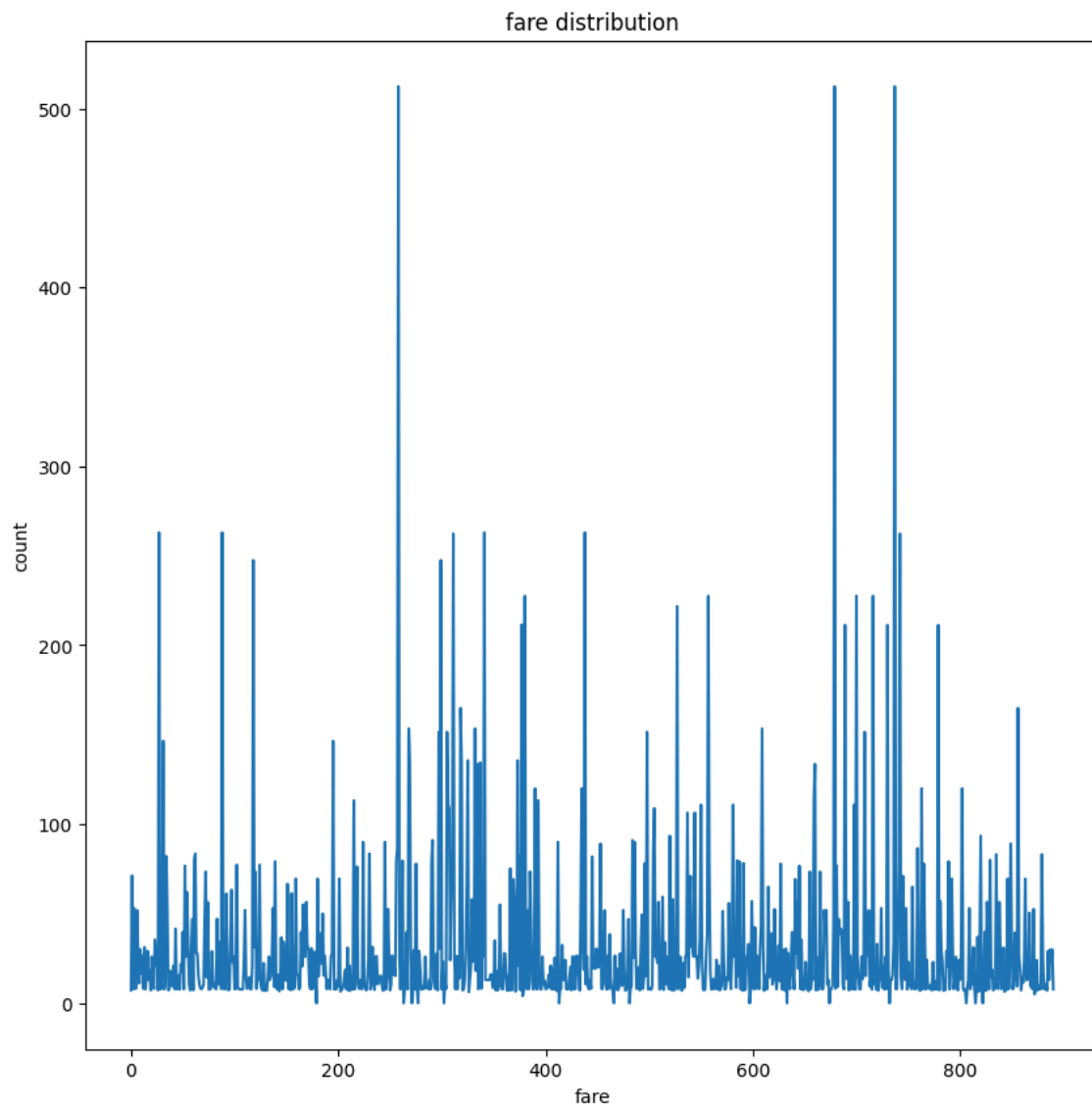
```

plt.figure(figsize=(10,10))
plt.hist(df['age'],bins=20)
plt.xlabel('age')
plt.ylabel('count')
plt.title('age distribution')
plt.show()

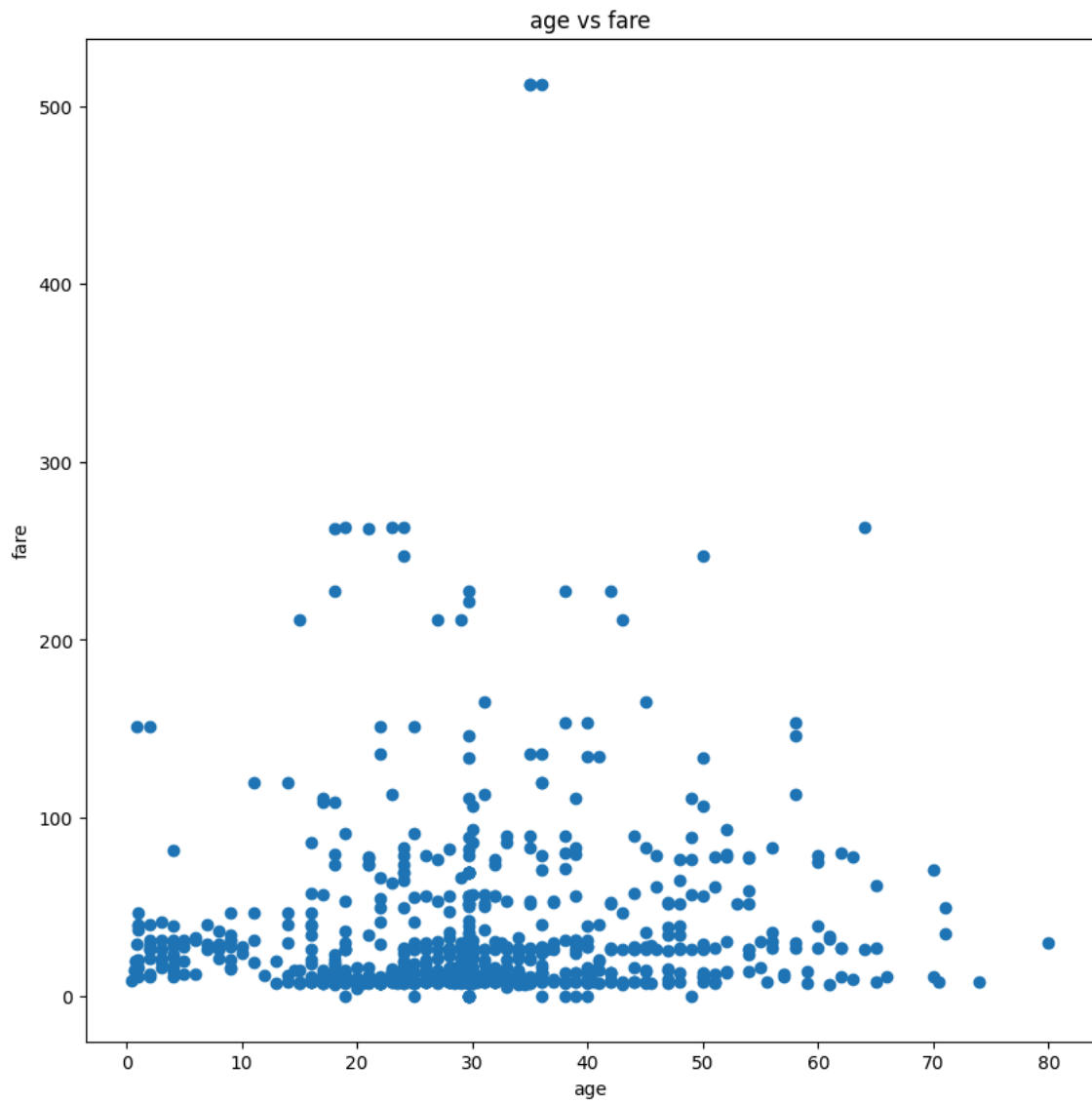
```



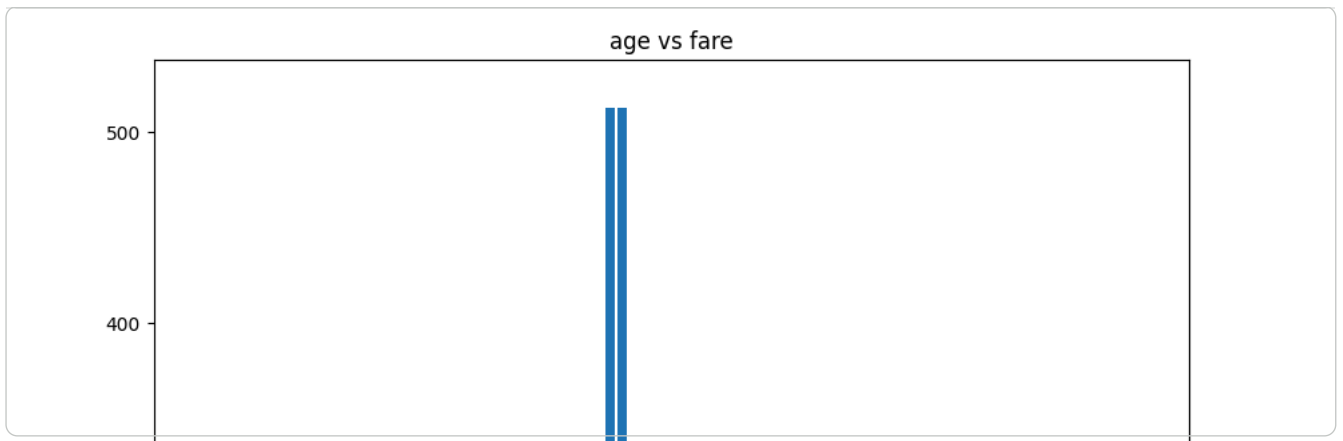
```
plt.figure(figsize=(10,10))
plt.plot(df['fare'])
plt.xlabel('fare')
plt.ylabel('count')
plt.title('fare distribution')
plt.show()
```



```
plt.figure(figsize=(10,10))
plt.scatter(df['age'],df['fare'])
plt.xlabel('age')
plt.ylabel('fare')
plt.title('age vs fare')
plt.show()
#
```



```
plt.figure(figsize=(10,10))
plt.bar(df['age'],df['fare'])
plt.xlabel('age')
plt.ylabel('fare')
plt.title('age vs fare')
plt.show()
```



```
plt.figure(figsize=(10,10))
```

